LABORATORY TITLE: INVESTIGATING RHION POSSESSION EVIDENCE

STUDENT NAME: WALIYAT OLALEKAN

STUDENT ID: 2025/FWSD/11435

COURSE NAME: COMPUTER FORENSICS CASE STUDY

COURSE CODE: SBT-DF204 (LAB 4)

INSTRUCTOR NAME: MR. AMINU IDRIS

DATE OF SUBMISSION: OCTOBER 26TH , 2026.

VERSION: Version 1.0

Executive Conclusion

This forensic examination analyzed network packet captures (rhino.log, rhino2.log, rhino3.log) and a raw USB disk image (RHINOUSB.dd). Cryptographic hash verification, file system enumeration via The Sleuth Kit, and packet stream reconstruction in Wireshark confirm structured file transfers, file carving recoveries, and archive extraction activities. The technical findings - including cryptographic hash verification, file-system recovery, and network stream reconstruction are established with a moderate-to-high degree of confidence. However, conclusions regarding human attribution, physical device possession, or user intent are maintained at a low confidence level, as digital artifacts and technical logs record system and network events rather than individual human agency.

Evidence Acquisition and Integrity

The original evidence (rhino.log, rhino2.log, rhino3.log and RHINOUSB.dd) were secured by hashing them using sha-256 checksum, creating working copies and implementing chmod 444 on the original copy prior to examination.

Filename	Size	Sha-256 checksum
rhino.log	3.1MB	64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2
rhino2.log	286KB	41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde
rhino3.log	221KB	7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46
RHINOUSB.dd	248MB	ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65

Methodology

The examination was conducted in a controlled, isolated Kali Linux environment following systematic digital forensics principles to ensure reproducibility, data integrity, and defensibility.

Prior to any interaction, baseline SHA-256 cryptographic hashes were computed for all original case artifacts (RHINOUSB.dd, rhino.log, rhino2.log, and rhino3.log) to establish a strict chain of custody. Independent working duplicates were generated for all evidence files. Strict file-system permissions (chmod 444) were applied to the original evidence objects to enforce read-only protection and prevent accidental modification or spoliation.

Initial structural parsing via partition table queries (mmls) yielded no output, confirming that the target image was structured as a raw volume initiating directly at sector offset 0. Structural analysis was executed using The Sleuth Kit (fsstat with offset 0 and the -f fat flag), identifying an intact FAT16 file system layout with a 512-byte sector and 4,096-byte cluster configuration. Directory structures and allocation states were indexed using fls, locating active entries (such as gumbo1.txt and gumbo2.txt) for targeted file extraction via inode referencing (icat).

Unallocated and fragmented space across the raw volume was processed using the PhotoRec carving suite to recover deleted media and document assets. Recovered outputs were filtered into distinct artifact groups (JPEGs, GIFs, and DOC files). Cryptographic SHA-256 hashes were generated for all targeted files to catalog unique objects and isolate duplicates. Encapsulated archives (contraband.zip) discovered during the examination were subjected to authorized dictionary-based password recovery workflows utilizing fcrackzip against standard wordlist parameters (rockyou.txt). Upon successful key recovery, encapsulated contents (rhino2.jpg) were

unpacked, visually inspected, and integrated into the cryptographic hashing matrix for cross-verification.

Packet captures (rhino.log) were parsed in Wireshark utilizing protocol filters (ftp, ftp-data). TCP stream reconstruction (Stream 69) was performed to extract control session authentication exchanges and trace data object transfers (STOR rhino1.jpg). HTTP traffic logs (rhino2.log and rhino3.log) were analyzed via request filtering and automated object export functions to harvest web-transferred assets (including graphic elements and executable binaries). External signature correlation was used to profile extracted binaries (e.g., matching hashes to diskp.exe / rhino.exe).

Automated file-comparison workflows (comm, grep) were executed to cross-reference the cryptographic hashes of USB-carved files against network-extracted objects, establishing objective data parity where applicable. File system metadata (MAC times) and packet timestamps were normalized and aligned into a unified UTC chronological sequence to reconstruct the investigative event chain.

Investigation Questions

1. How did you preserve the case materials?

I preserved the case materials by creating isolating working copies (e.g. rhino.log_working.raw). I applied read only permissions (chmod 444) to the original evidence files to guarantee integrity and prevent modification or tampering. I then generated sha-256 checksums for all the original files and saved them to a hashes text file.

2. What does the USB disk image show before recovery?

Before recovery, I ran mmls command and it returned no partition table because the image is a raw volume initiating directly at sector offset 0. The fsstat -o 0 command confirmed a FAT16 file system with a sector size of 512 bytes and a cluster size of 4,096 bytes. Running fls -o 0 -r exposed allocated file entries including gumbo1.txt and gumbo2.txt (inode 4 and 6 respectively). Extracting these via icat revealed an active text recipe for "SHRIMP AND ANDOUILLE SAUSAGE GUMBO."

3. What deleted or recoverable files are supported?

Using Photorec, a directory (recup_dir.2) containing recovered items was generated. Going through the directory, 7 JPEGs, 3 GIFs and 1 DOC file was found amongst several text files. Cryptographic hashing (sha256sum) was performed on targeted items to catalog and isolate unique versus duplicate files for the evidence log.

4. Is there evidence of hidden content in the recovered media?

Analysis of archive elements identified contraband.zip. Utilizing an authorized wordlist attack (fcrackzip -u -D -p rockyou.txt contraband.zip), the password for the archive was successfully recovered (pw = monkey). Unzipping the archive extracted an embedded image (rhino2.jpg, extracted as newrhino2.jpg), confirming hidden/protected archive contents which were subsequently hashed and visually inspected.

5. What does the FTP capture establish?

Filtering for FTP traffic isolated communication between client 137.30.122.40 and server 137.30.122.253 (TCP Stream 69). Authentication records exposed explicit control session credentials (USER gnome, PASS gnome123). Command logs verified a file transfer via the STOR command (STOR rhino1.jpg), successfully transferring 65,703 bytes. Visual inspection using ImageMagick (display rhino1.jpg) confirmed the object is a photograph of a rhinoceros.

6. What does the protected archive analysis establish?

The analysis of contraband.zip proved that password protection was utilized to encapsulate sensitive media (rhino2.jpg). Following the authorized laboratory password-recovery workflow (monkey), the archive was unpacked. Hashing the recovered file (rhino.jpg / newrhino2.jpg) established binary integrity and allowed direct comparison against other dataset objects.

7. What does the HTTP capture establish?

Reviewing exported HTTP objects identified several graphic elements (e.g., rhino5.gif, logo.gif) and a binary executable file named rhino.exe (145 KB, application/octet-stream). Inspecting specific packet flows (http.content_type[0:5] == "image" and http contains "gif") confirmed web-based asset retrieval from endpoint 137.30.122.234. External signature/hash correlation queries mapped the binary string hash to diskp.exe (a renamed variant of the Windows DiskPart utility).

8. What conclusion can you defend?

Cryptographic hashing and stream analysis conclusively prove that specific files (rhino1.jpg, rhino2.jpg, rhino.exe) were stored on the USB volume, transmitted via FTP sessions, and retrieved over HTTP channels.

While technical identifiers (IP addresses, filenames, and hash matches) corroborate network and media activity, they do not prove physical possession of the hardware or identify the human operator at the keyboard. Alternative explanations (such as automated script syncs or shared technical accounts) cannot be ruled out.

Timeline template

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
Media preservation	2004-04-26 23:13 UTC	E01	Original binaries	Hashes computed; read-only permissions applied	Chain of custody secured.
Disk image / file-system review	2004-04-26 23:14 UTC	E02	RHINOUSB.dd	Identified FAT16 structure and located deleted inodes.	Raw volume layout verified
Recovered or hidden content	2004-04-26 23:15 UTC	E03	Inode 4, 6 and Photorec	Extracted text files and carved media set	Proves file deletion
FTP transfer activity	2004-04-26 18:22 UTC	E04	rhino.log	FTP session recorded use gnome storing image.	Network transfer corroborated
HTTP transfer activity	2004-04-28 21:07 UTC	E05	rhino2.log	Transmitted HTTP objects and executable rhino.exe	Web artifact downloaded

Corroborated conclusion	Comparative analysis	All	Linked network captures with local storage artefacts	Technical link established human attribution unproven	

Evidence log template

Evidence ID	Tool and method	Source record, stream, or path	Observed path	Screenshot or appendix reference
E01	Sha-256 sum	Working directory	Generated baseline hashes for original evidence files.	Preserve and identify evidence.
E02	The Sleuth Kit (fsstat,fls)	RHINOUSB.dd (offset o)	Confirmed FAT16 volume layout, listed deleted inodes 4 and 6	Examine the disk image.
E03	The Sleuth Kit (icat)	Inode 4 and 6	Extracted deleted recipe text file	Examine the disk image.
E04	Photorec	recup_dir.2	Discovered 7 jpegs, 3 gifs and 1 doc file	Recover deleted files.
E05	frackzip and unzip	contraband.zip	Cracked password (monkey); extracted newrhino2.jpg	Review media and hidden data.
E06	Wireshark (TCP stream)	rhino.log	Captured FTP login for user gnome and storage of rhino1.jpg	Review FTP traffic.
E07	Wireshark (export HTTP)	rhino2.log	Exported rhino.exe	Review HTTP traffic.

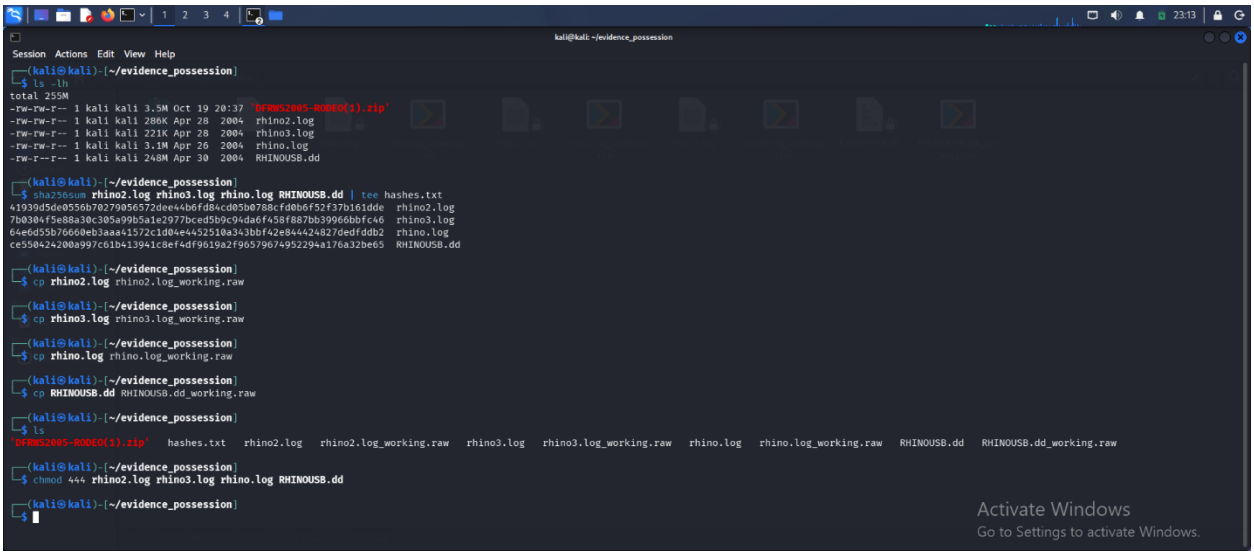
Conclusion and Limitations

The investigation successfully established that digital assets were stored on the USB image and transmitted across FTP and HTTP channels. However, several limitations must be noted:

- Timestamp Discrepancies: Log timestamps date back to 2004, requiring careful synchronization during analysis.
- Attribution Gap: While account credentials (user gnome) and file hashes establish network activity and storage presence, they do not provide conclusive proof of physical human identity at the keyboard.
- Malware Safety: Reconstructed executable objects (rhino.exe) were identified via threat intelligence databases as system administration binaries (DiskPart equivalents) and were isolated without execution.

Screenshots/Appendices

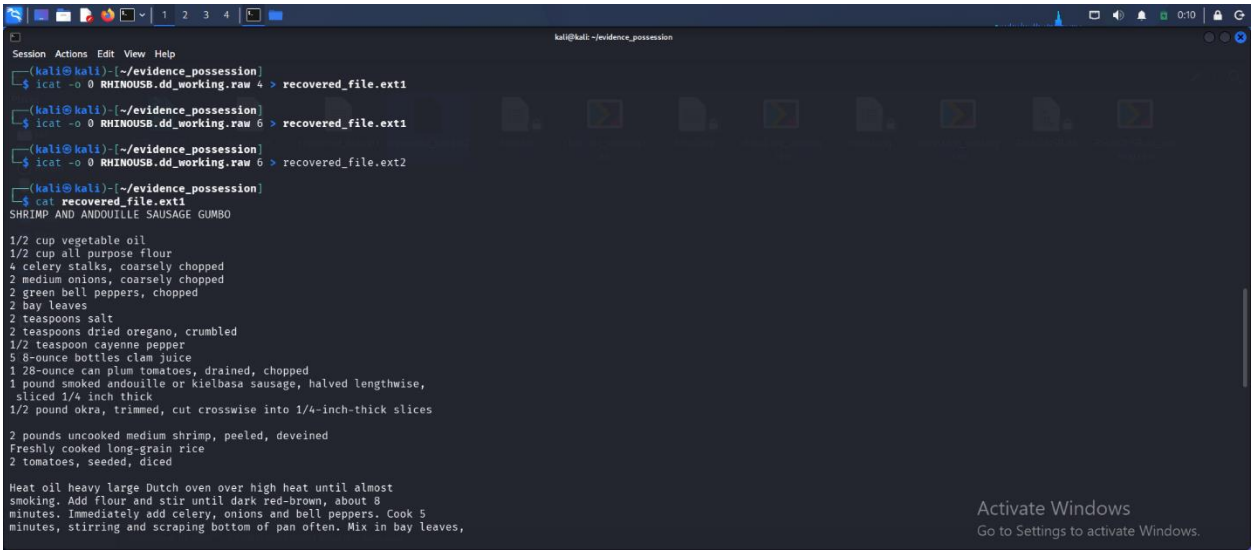
Preserve and identify evidence



Examine the disk image

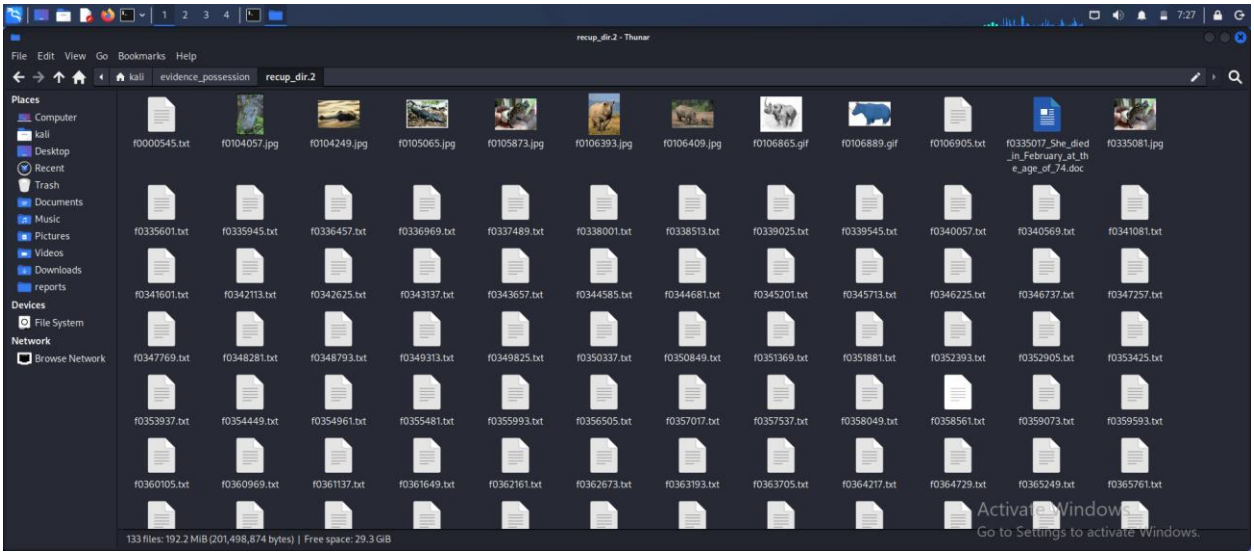




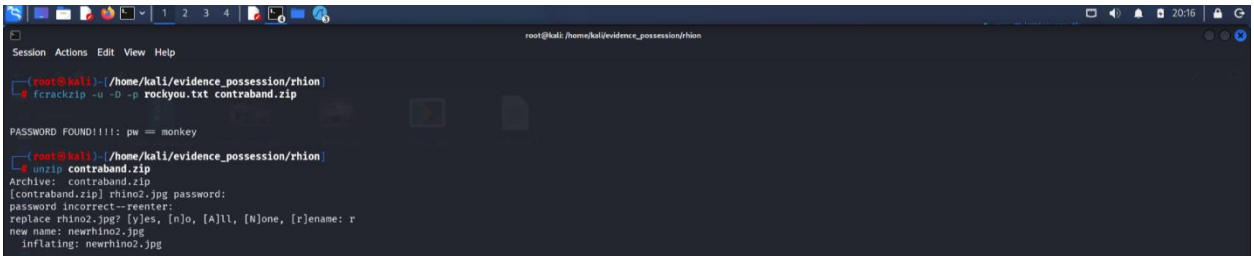




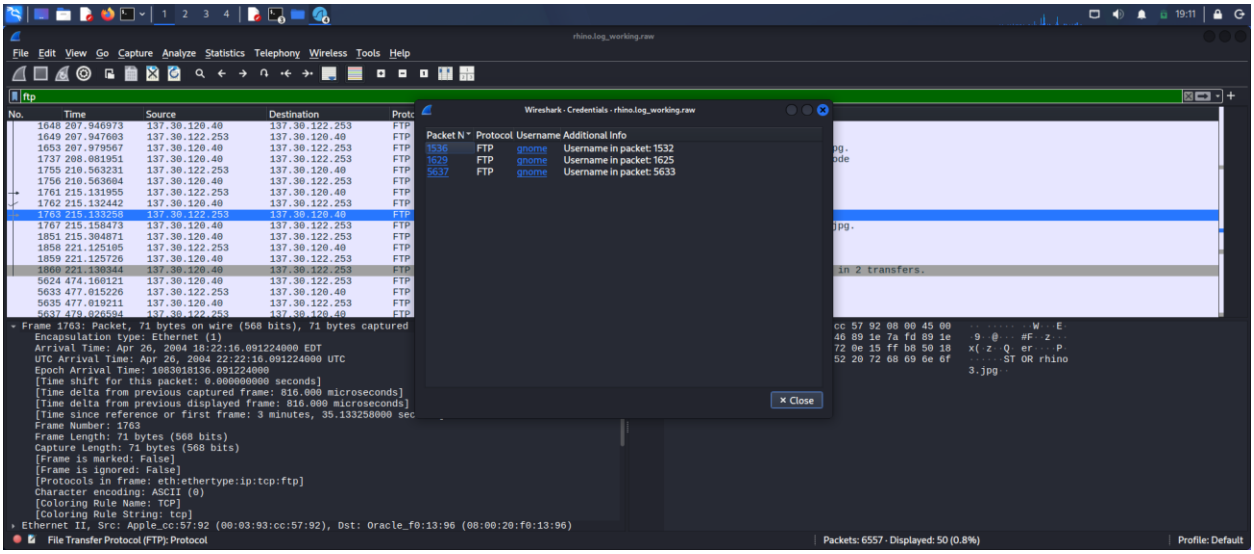
Recover deleted files

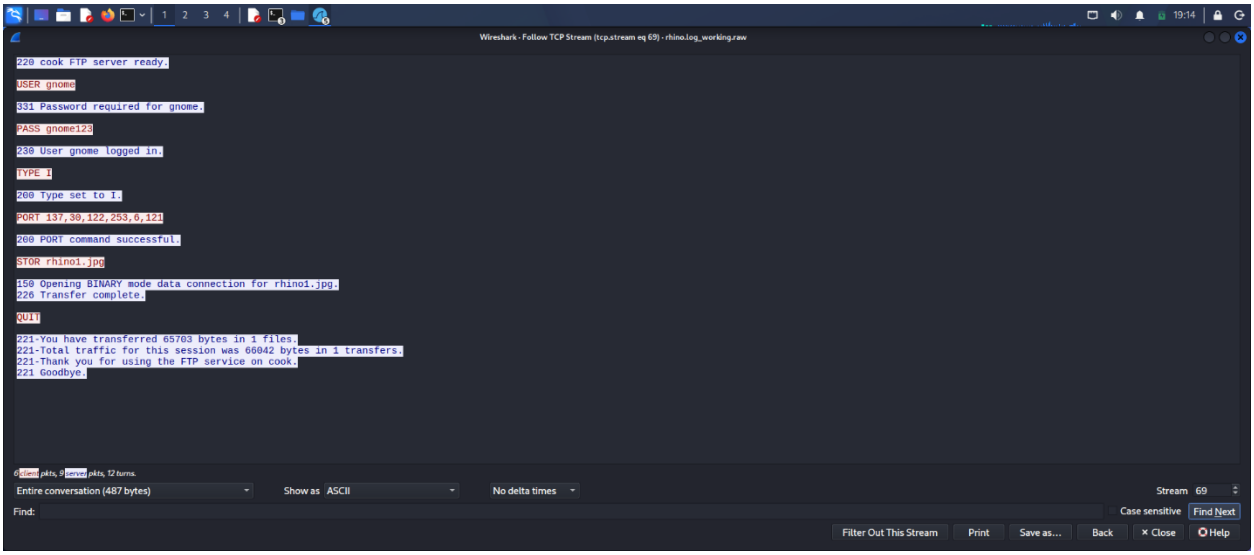


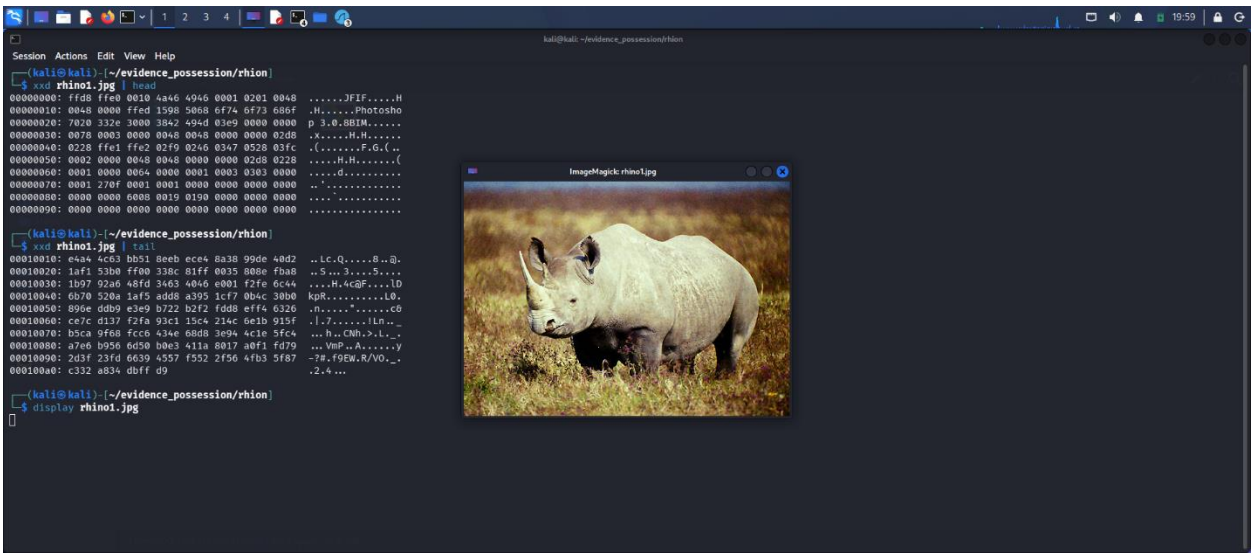
Review media and hidden data

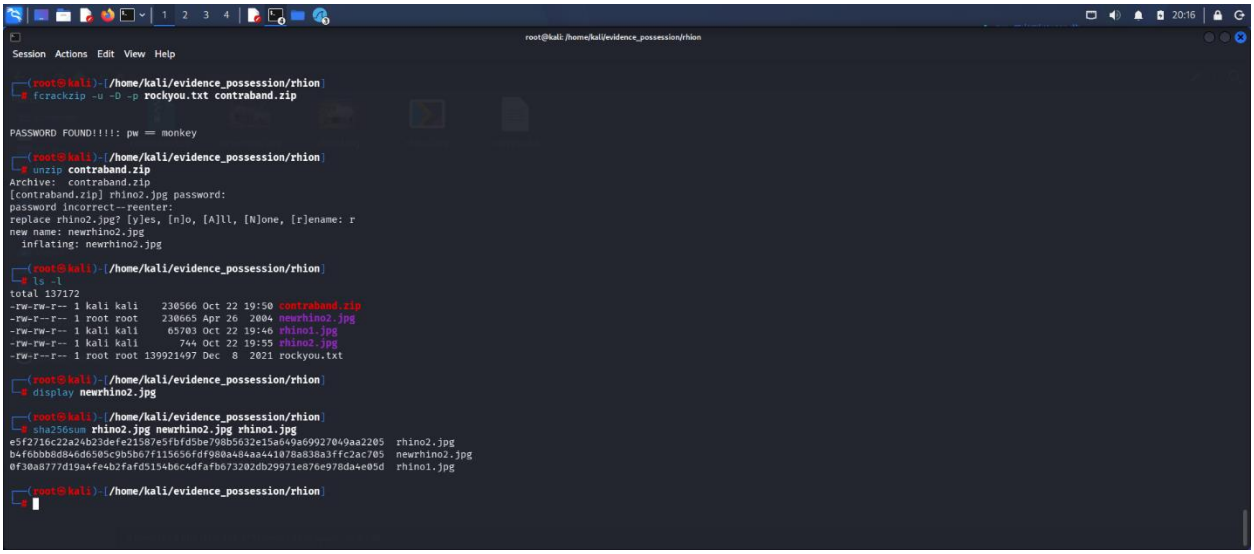


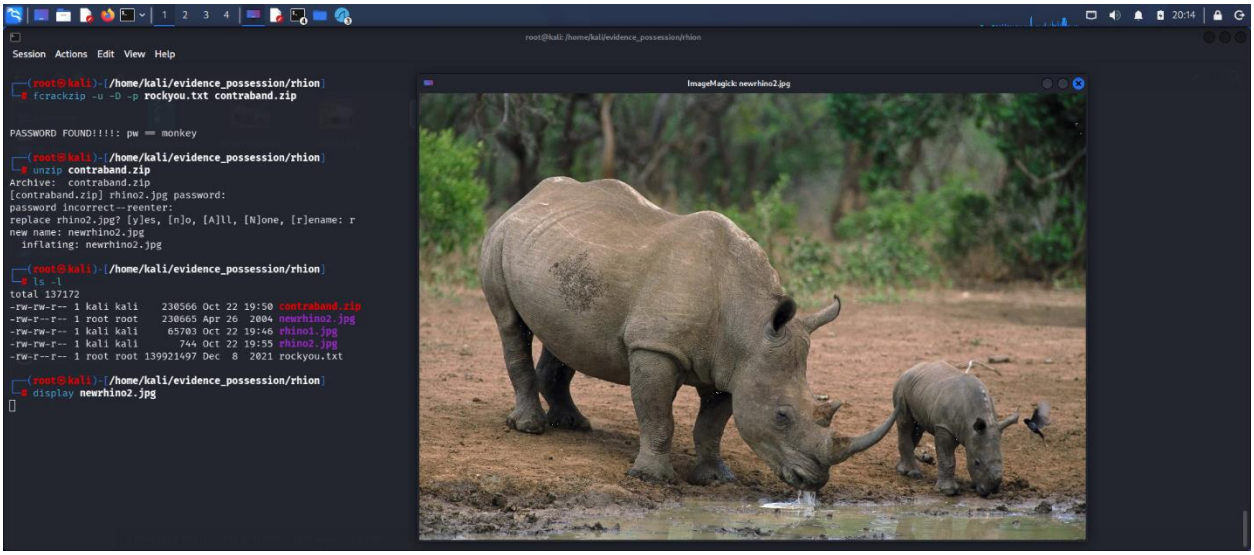
Review FTP traffic



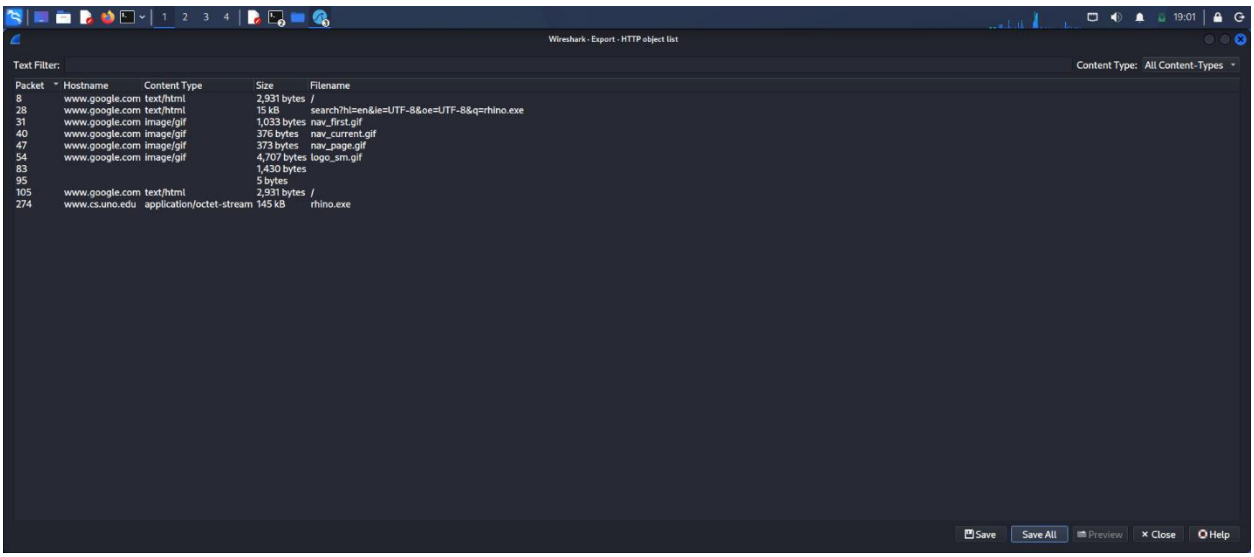


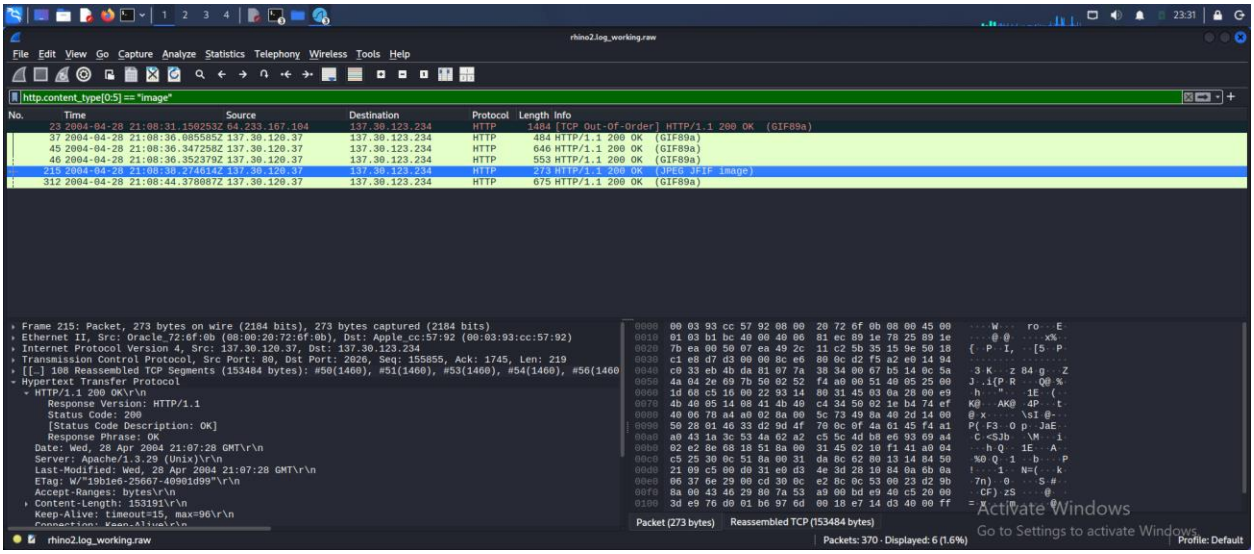


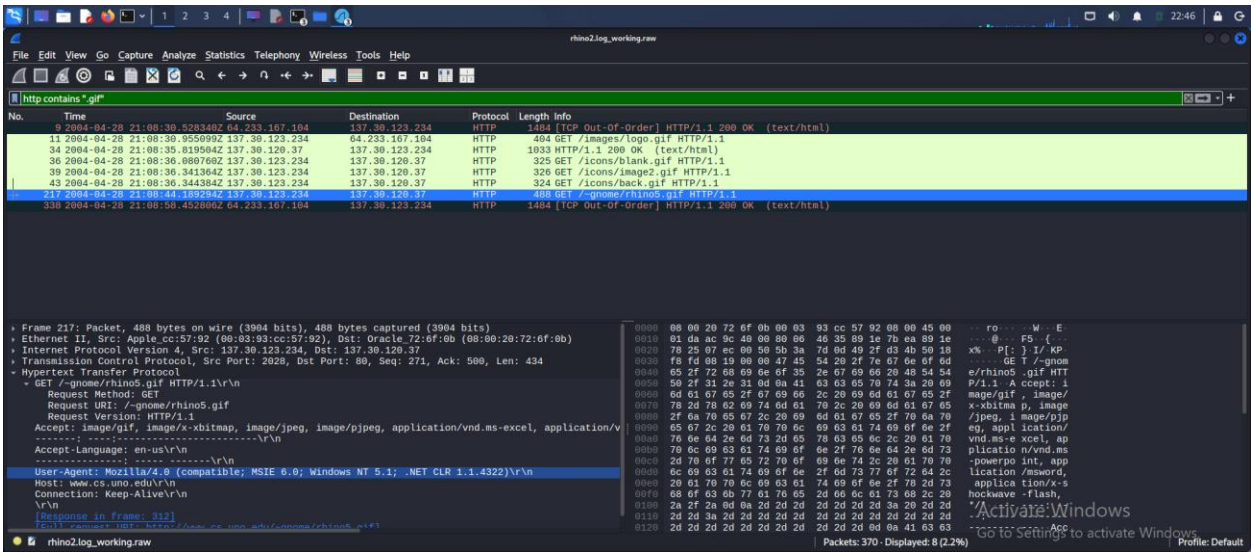


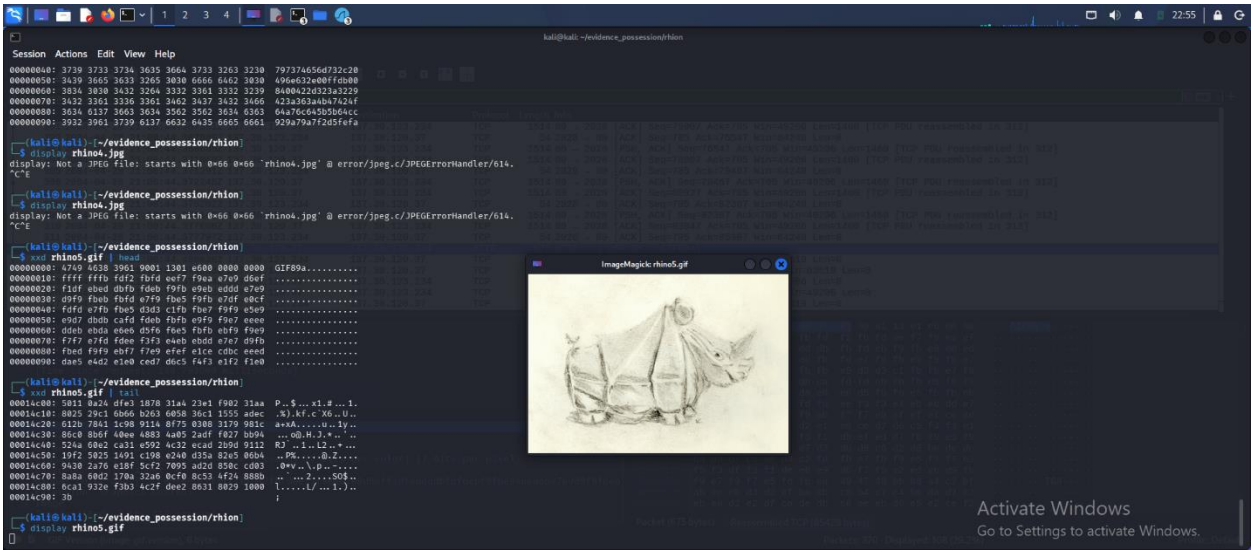


Review HTTP traffic

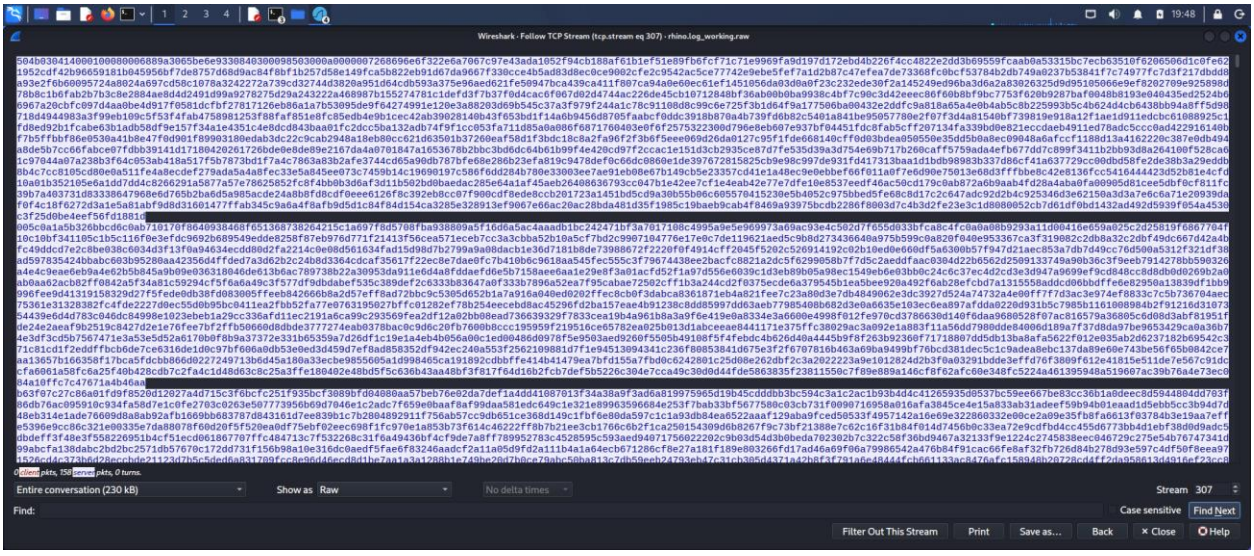


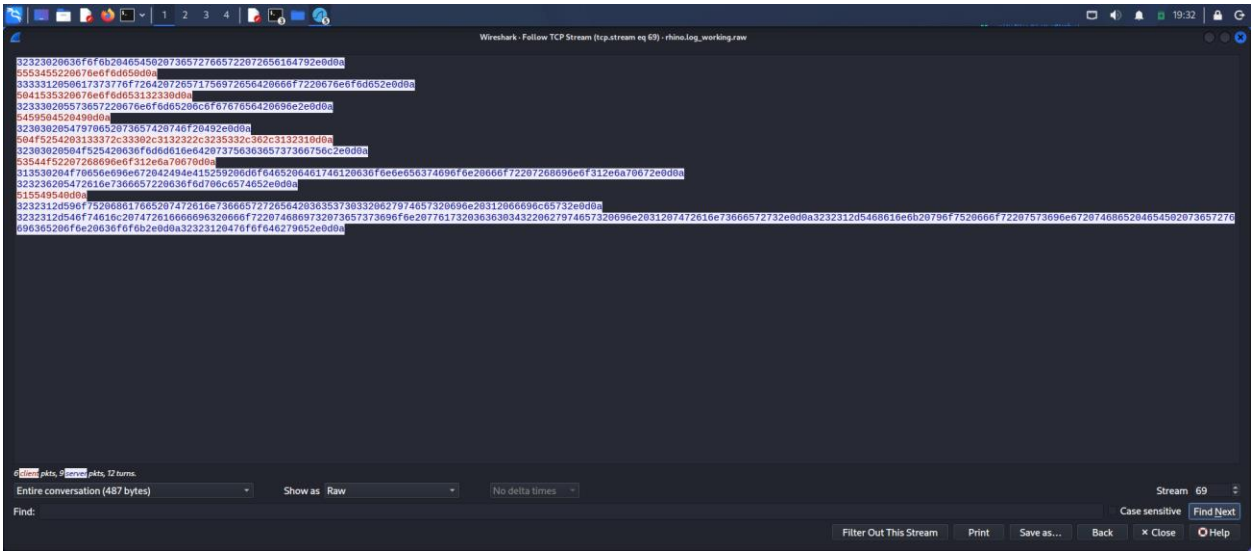

rhino2.log_working.raw
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
http.content_type[0:5] == "image"
No. Time Source Destination Protocol Length Info
23 2004-04-28 21:08:31.150253Z 64.233.167.104 137.30.123.234 HTTP 1484 [TCP Out-Of-Order] HTTP/1.1 200 OK (GIF89a)
37 2004-04-28 21:08:36.085585Z 137.30.120.37 137.30.123.234 HTTP 484 HTTP/1.1 200 OK (GIF89a)
45 2004-04-28 21:08:36.347258Z 137.30.120.37 137.30.123.234 HTTP 646 HTTP/1.1 200 OK (GIF89a)
46 2004-04-28 21:08:36.352379Z 137.30.120.37 137.30.123.234 HTTP 553 HTTP/1.1 200 OK (GIF89a)
215 2004-04-28 21:08:50.274014Z 137.30.120.37 137.30.123.234 HTTP 273 HTTP/1.1 200 OK (JPEG JFIF image)
312 2004-04-28 21:08:44.378087Z 137.30.120.37 137.30.123.234 HTTP 675 HTTP/1.1 200 OK (GIF89a)
Frame 215: Packet, 273 bytes on wire (2184 bits), 273 bytes captured (2184 bits)
Ethernet II, Src: Oracle_72:6f:0b (08:00:20:72:6f:0b), Dst: Apple_cc:57:92 (00:03:93:cc:57:92)
Internet Protocol Version 4, Src: 137.30.120.37, Dst: 137.30.123.234
Transmission Control Protocol, Src Port: 80, Dst Port: 2026, Seq: 155855, Ack: 1745, Len: 219
[[…] 108 Reassembled TCP Segments (153484 bytes): #50(1460), #51(1460), #53(1460), #54(1460), #56(1460
Hypertext Transfer Protocol
HTTP/1.1 200 OK\r\n
Response Version: HTTP/1.1
Status Code: 200
[Status Code Description: OK]
Response Phrase: OK
Date: Wed, 28 Apr 2004 21:07:28 GMT\r\n
Server: Apache/1.3.29 (Unix)\r\n
Last-Modified: Wed, 28 Apr 2004 21:07:28 GMT\r\n
ETag: W/"19b1e6-25607-40901d99"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 153191\r\n
Keep-Alive: timeout=15, max=96\r\n
Packet (273 bytes)
Reassembled TCP (153484 bytes)
Packets: 370 · Displayed: 6 (1.6%)
Profile: Default
Activate Windows
Go to Settings to activate Windows.

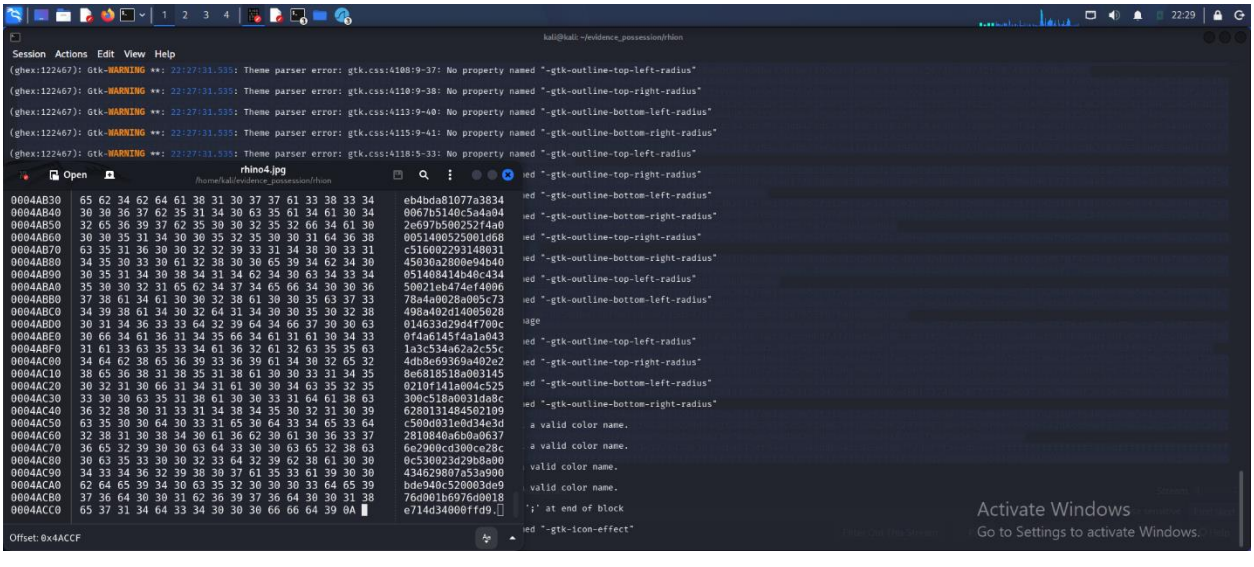

rhino2.log_working.raw
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
http contains ".gif"
No. Time Source Destination Protocol Length Info
9 2004-04-28 21:08:30.528340Z 64.233.167.104 137.30.123.234 HTTP 1484 [TCP Out-Of-Order] HTTP/1.1 200 OK (text/html)
11 2004-04-28 21:08:30.955099Z 137.30.123.234 64.233.167.104 HTTP 404 GET /images/logo.gif HTTP/1.1
34 2004-04-28 21:08:35.819504Z 137.30.120.37 137.30.123.234 HTTP 1033 HTTP/1.1 200 OK (text/html)
36 2004-04-28 21:08:36.080760Z 137.30.123.234 137.30.120.37 HTTP 325 GET /icons/blank.gif HTTP/1.1
39 2004-04-28 21:08:36.341364Z 137.30.123.234 137.30.120.37 HTTP 326 GET /icons/image2.gif HTTP/1.1
43 2004-04-28 21:08:36.344384Z 137.30.123.234 137.30.120.37 HTTP 324 GET /icons/back.gif HTTP/1.1
217 2004-04-28 21:08:44.189294Z 137.30.123.234 137.30.120.37 HTTP 488 GET /~gnome/rhino5.gif HTTP/1.1
338 2004-04-28 21:08:58.452806Z 64.233.167.104 137.30.123.234 HTTP 1484 [TCP Out-Of-Order] HTTP/1.1 200 OK (text/html)
Frame 217: Packet, 488 bytes on wire (3904 bits), 488 bytes captured (3904 bits)
Ethernet II, Src: Apple_cc:57:92 (00:03:93:cc:57:92), Dst: Oracle_72:6f:0b (08:00:20:72:6f:0b)
Internet Protocol Version 4, Src: 137.30.123.234, Dst: 137.30.120.37
Transmission Control Protocol, Src Port: 2028, Dst Port: 80, Seq: 271, Ack: 500, Len: 434
Hypertext Transfer Protocol
GET /~gnome/rhino5.gif HTTP/1.1\r\n
Request Method: GET
Request URI: /~gnome/rhino5.gif
Request Version: HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, application/vnd.ms-excel, application/v
Accept-Language: en-us\r\n
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; .NET CLR 1.1.4322)\r\n
Host: www.cs.uno.edu\r\n
Connection: Keep-Alive\r\n
\r\n
[Response in frame: 312]
rhino2.log_working.raw
Packets: 370 · Displayed: 8 (2.2%)
Profile: Default
Activate Windows
Go to Settings to activate Windows.

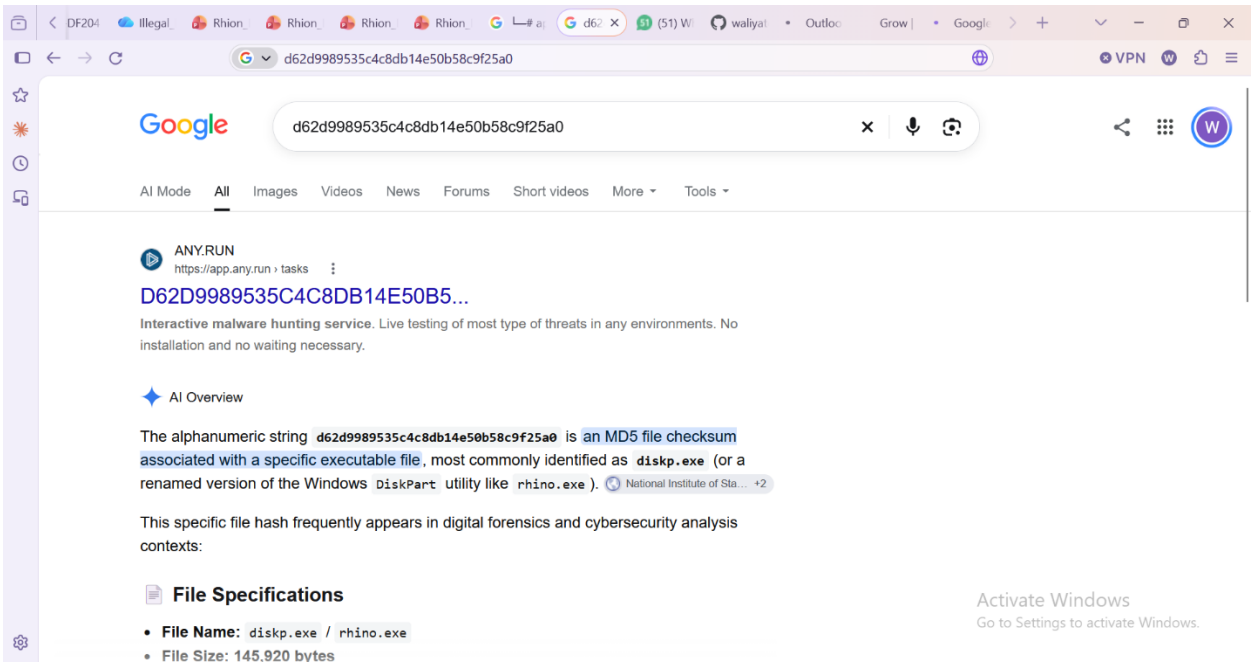

Session Actions Edit View Help
display: Not a JPEG file: starts with 0×66 0×66 `rhino4.jpg' @ error/jpeg.c/JPEGErrorHandler/614.
display: Not a JPEG file: starts with 0×66 0×66 `rhino4.jpg' @ error/jpeg.c/JPEGErrorHandler/614.
ImageMagick: rhino5.gif
Activate Windows
Go to Settings to activate Windows.

Validate reconstructed objects








d62d9989535c4c8db14e50b58c9f25a0
Google
d62d9989535c4c8db14e50b58c9f25a0
AI Mode
All
Images
Videos
News
Forums
Short videos
More
Tools
ANY.RUN
https://app.any.run › tasks
D62D9989535C4C8DB14E50B5...
Interactive malware hunting service. Live testing of most type of threats in any environments. No installation and no waiting necessary.
AI Overview
The alphanumeric string d62d9989535c4c8db14e50b58c9f25a0 is an MD5 file checksum associated with a specific executable file, most commonly identified as diskp.exe (or a renamed version of the Windows DiskPart utility like rhino.exe). National Institute of Sta... +2
This specific file hash frequently appears in digital forensics and cybersecurity analysis contexts:
File Specifications
File Name: diskp.exe / rhino.exe
File Size: 145,920 bytes
Activate Windows
Go to Settings to activate Windows.